Projet MLSF : résolution du jeu Meowdoku

Question 6 : conception et évaluation du solveur optimisé

1 Choix de conception

La question 5 a mis en évidence deux résultats : H8, qui privilégie les littéraux positifs, est adaptée à Meowdoku ; le recalcul d'une heuristique sur toute la formule peut coûter plus que le gain obtenu dans l'arbre de recherche. Le mode `optimized` de `sat.py` appelle désormais un moteur distinct, défini dans `optimized.py`. Le mode `dpll` conserve l'algorithme 2 de la question 2. Le nouveau moteur reste un DPLL complet, sans apprentissage de clauses ni dépendance externe.

1.1 Propager sans recopier la formule

Le DPLL initial reconstruit la liste des clauses à chaque affectation. Dans le moteur optimisé, les clauses sont conservées et les affectations sont enregistrées dans un tableau de valeurs et une pile chronologique (*trail*). Une file implicite, repérée par un indice dans cette pile, contient les affectations dont les conséquences restent à propager.

Clauses binaires. Une clause $(a \vee b)$ est représentée par les implications $\neg a \Rightarrow b$ et $\neg b \Rightarrow a$. Fixer un littéral à vrai ne visite donc que ses conséquences. Cette représentation convient particulièrement aux nombreuses contraintes Meowdoku $(\neg x_{i,j} \vee \neg x_{k,l})$: placer un chat exclut directement les cases incompatibles.

Clauses d'au moins trois littéraux. Deux littéraux sont surveillés. Lorsqu'un littéral surveillé devient faux, on cherche un remplaçant non faux. S'il n'en existe pas, l'autre surveillance est soit vraie (clause satisfaite), soit libre (propagation unitaire), soit fausse (conflit). Les clauses qui ne sont pas touchées ne sont pas parcourues. Les clauses unitaires initiales sont affectées avant la recherche ; une clause vide entraîne immédiatement UNSAT.

Retour arrière. Chaque niveau mémorise sa décision et la position de début dans le *trail*. Un échec annule seulement les affectations de ce niveau, puis essaie la polarité opposée. La recherche est itérative : elle ne dépend pas de la limite de récursion de Python. Les tautologies et les clauses identiques sont éliminées au prétraitement. La règle du littéral pur n'est pas recherchée pendant la résolution, pour éviter un parcours global supplémentaire ; elle n'est pas nécessaire à la correction de DPLL.

1.2 Une heuristique adaptée à la forme de la CNF

Le prétraitement reconnaît une classe syntaxique : toutes les clauses sont soit entièrement positives, soit des exclusions binaires négatives. L'encodage Meowdoku de la question 1 appartient à cette classe, mais le test ne repose ni sur le nom du fichier ni sur une taille de grille.

Pour cette classe, on choisit la clause positive non satisfaite qui a le moins de candidats encore libres, puis on essaie un candidat à vrai. Les égalités entre clauses suivent leur ordre d'entrée ; au sein d'une clause, les candidats sont départagés par le score positif initial $J^+(x) = \sum_{C \ni x} 2^{-|C|}$, puis par le plus petit numéro de variable. C'est une adaptation de H8 : la priorité aux clauses positives courtes est actualisée, mais seules les clauses positives sont parcourues. Sur Meowdoku, il s'agit des clauses imposant au moins un chat par région, beaucoup moins nombreuses que les exclusions. Ce choix n'est pas strictement H8d lorsque plusieurs clauses positives se recouvrent.

Pour les autres CNF, le moteur utilise H4d : il compte les occurrences dans les clauses résiduelles de taille minimale, choisit la variable la plus fréquente et essaie sa polarité majoritaire. Les égalités sont départagées par le plus petit numéro de variable, puis par la polarité positive.

2 Correction, interface et validation

2.1 Invariants de correction

Les implications binaires sont équivalentes à leurs clauses d'origine. Les surveillances détectent toute clause devenue unitaire ou contradictoire avant la prochaine décision. Au retour arrière, une variable passe de fixée à libre : les surveillances n'ont pas à être restaurées, car aucune nouvelle valeur fausse n'est introduite. La position de propagation est replacée au début du niveau annulé.

Le branchement explore toujours les deux valeurs d'une variable libre. Un conflit à la racine, ou l'épuisement des deux branches à chaque niveau, permet de conclure UNSAT. Pour une CNF générale, SAT n'est renvoyé qu'une fois toutes les clauses satisfaites. Pour la classe de placement, il suffit que toutes les clauses positives soient satisfaites : les implications ont déjà interdit deux valeurs vraies incompatibles, et compléter les variables restantes à faux préserve les exclusions. Le modèle retourné affecte toutes les variables déclarées, même absentes des clauses. Ces arguments ne supposent pas que la grille soit solvable.

Coût et limites algorithmiques. Le stockage des clauses, implications, surveillances et affectations est linéaire dans la taille de la CNF, hors surcoût des objets Python. Le tri et la déduplication sont effectués une fois. Une propagation ne recopie pas la CNF ; elle visite les implications et clauses concernées. Une clause longue peut toutefois être parcourue plusieurs fois. La recherche reste exponentielle dans le pire cas ; les surveillances réduisent le coût des nœuds, pas nécessairement leur nombre. H4d parcourt encore la formule à chaque décision.

2.2 Interface imposée

```
python sat.py instance.cnf --mode dpll
python sat.py instance.cnf --mode optimized --output-model
```

Par défaut, le mode est `optimized`. La sortie standard contient `s SATISFIABLE` ou `s UNSATISFIABLE`, et, pour SAT avec `-output-model`, une ligne `v ... 0`. Aucun modèle n'est écrit pour UNSAT.

L'option facultative `-stats` écrit sur **stderr** un objet JSON contenant les durées et les compteurs. Elle ne modifie pas le format de sortie imposé. Les décisions comptent les nouveaux branchements ; les retours arrière comptent le passage à la polarité opposée ; les propagations comptent les nouvelles affectations forcées, unités initiales comprises. Le compteur de littéraux purs vaut zéro dans le moteur optimisé. Le temps de l'heuristique exclut le prétraitement et inclut la dernière vérification de terminaison. L'option `-seed` fixe les choix aléatoires du mode DPLL.

2.3 Vérifications effectuées

— Les tests du projet et les nouveaux tests passent : **54 tests**.
— Le moteur optimisé est comparé à un oracle exhaustif sur **2 000 CNF aléatoires** de 1 à 9 variables, dont des CNF de placement. Chaque modèle SAT est vérifié clause par clause.
— Des tests ciblent les clauses vides, formules vides, tautologies, doublons, unités contradictoires, déplacements de surveillance suivis de retours arrière, ainsi que des recherches et propagations profondes.
— Les tests d'interface contrôlent les deux modes, la complétude du modèle et la séparation stdout/stderr. Le banc de mesure rejette un modèle faux, incomplet, répété ou hors limites et une sortie parasite.

Les tests augmentent la confiance dans l'implémentation ; ils ne remplacent pas les invariants de correction ci-dessus.

3 Évaluation expérimentale

3.1 Protocole reproductible

`bench_q6.py` lance réellement `sat.py` dans un nouveau processus par résolution, avec les options `-output-model` et `-stats`. Le délai est fixé à **60 secondes par processus**, et les lancements sont séquentiels. Le temps principal inclut le démarrage de Python, les imports, la lecture, la résolution et l'écriture du résultat. Le temps CPU interne inclut le prétraitement et la résolution, mais exclut les entrées et sorties. La vérification indépendante du modèle intervient ensuite, hors mesure.

Le corpus comprend **91 instances** : 30 benchmarks publics, 11 tests publics d'interface et de cas limites, 20 grilles Meowdoku SAT de tailles 8, 10, 12, 14 et 16 (quatre par taille), et 30 grilles à régions aléatoires (7 SAT, 23 UNSAT). Les 50 grilles et leurs graines sont celles de la question 5. Les statuts de référence proviennent du fichier public `expected.json`, de la construction des grilles SAT et, pour les régions aléatoires, de l'accord des heuristiques observé à la question 5.

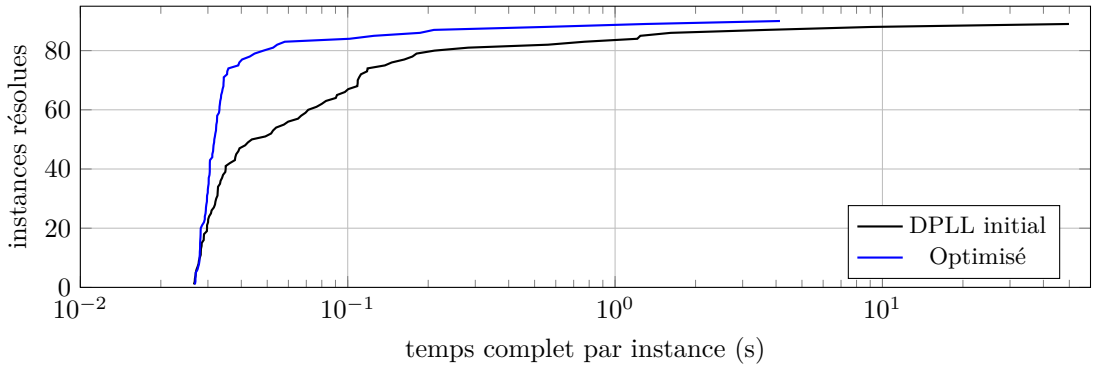
La campagne comporte une exécution par instance et par mode, soit **182 résolutions**, avec une graine DPLL dérivée du nom du fichier. Les mesures sont locales : Python 3.12.13, macOS 15.7.3, architecture ARM64. Elles ne sont pas des mesures sur la machine Linux x86_64 de la compétition. Le CSV `results/q6_runs.csv` conserve les résultats détaillés ; le fichier JSON associé décrit l'environnement et les paramètres.

3.2 Résultats globaux

Corpus	Mode	Résolus / essais	Temps total (s)	PAR-2 (s)
Ensemble	dpll	89/91	72.949	3.439
Ensemble	optimized	90/91	9.297	1.421
Benchmarks publics	dpll	28/30	69.624	10.321
Benchmarks publics	optimized	29/30	7.354	4.245
Interface et cas limites	dpll	11/11	0.317	0.029
Interface et cas limites	optimized	11/11	0.328	0.030
Meowdoku SAT	dpll	20/20	1.290	0.064
Meowdoku SAT	optimized	20/20	0.687	0.034
Régions aléatoires	dpll	30/30	1.718	0.057
Régions aléatoires	optimized	30/30	0.928	0.031

Le temps total ne porte que sur les résolutions correctes : il doit être lu avec leur nombre, qui est le premier critère de classement. Le PAR-2 est la moyenne des temps, avec une pénalité de 120 s pour chaque échec ou dépassement de délai. Il rend ces dépassements visibles dans une mesure globale ; il n'est pas le critère de classement imposé.

Le mode optimisé obtient 90/91 résolutions correctes, contre 89/91 pour DPLL. La campagne compte 0 erreur(s) de statut, de modèle ou d'interface. Dépassements de délai (dpll) : `public-03.cnf`, `public-06.cnf`. Dépassements de délai (optimized) : `public-03.cnf`. Sur les 89 paires résolues par les deux modes, la moyenne géométrique du rapport DPLL/optimisé vaut 1.81 pour le temps complet et 4.74 pour le temps CPU de résolution. Le gain n'est pas uniforme : sur `public-04.cnf`, le mode optimisé est 59.5 fois plus lent en temps complet pour cette graine de DPLL.



4 Interprétation pour Meowdoku

Taille	CPU DPLL (ms)	CPU optimisé (ms)	Processus DPLL (ms)	Processus optimisé (ms)
8	2.86	0.87	32.91	29.09
10	4.56	1.91	32.66	29.61
12	18.43	2.69	47.66	32.12
14	47.62	4.52	77.71	34.34
16	44.23	6.94	79.67	41.64

Ce tableau donne les temps médians sur les quatre grilles SAT de chaque taille. Les temps CPU isolent le travail du solveur ; les temps du processus correspondent davantage à l'expérience de la compétition.

Sur les 50 paires Meowdoku résolues par les deux modes, le gain géométrique est de **4.2 fois** sur le temps CPU du solveur, contre **1.5 fois** sur le temps complet. Le démarrage de Python et la lecture de la CNF réduisent donc le gain visible de bout en bout. Le mode optimisé réussit 50/50 résolutions Meowdoku.

Pourquoi le gain ? Une décision positive place un chat et déclenche directement les exclusions binaires de sa ligne, sa colonne, sa région et son voisinage. La formule n'est plus reconstruite pour chaque case exclue. Le choix de la région la plus contrainte ne parcourt que les clauses positives. Le coût d'actualisation reste ainsi limité, alors que les heuristiques dynamiques de la question 5 recalculaient leurs scores sur l'ensemble des clauses.

Le gain mesuré combine un nouveau moteur de propagation et une nouvelle heuristique ; ce protocole ne permet pas d'attribuer une part chiffrée à chacune. Il compare la solution finale à l'algorithme 2, sans réutiliser les temps anciens de la question 5 comme s'ils appartenaient à la même campagne.

Portée des résultats. Le DPLL de référence n'est exécuté qu'avec une graine par instance. Une campagne à plusieurs graines serait nécessaire pour quantifier la variabilité de ses temps. Les mesures très courtes sont également sensibles à la charge et à la fréquence du processeur. La campagne actuelle n'est ni une garantie de gain sur toute CNF, ni une estimation du classement sur les instances privées.

L'absence d'apprentissage de clauses limite les performances sur les instances exigeant de très nombreux retours arrière. Le mode général H4d conserve aussi un coût de parcours global par décision. Les grilles à régions aléatoires du corpus sont majoritairement faciles ; des Meowdoku UNSAT plus difficiles et des tailles supérieures permettraient d'évaluer plus largement la robustesse du solveur.

5 Reproduire les résultats

Depuis la racine du projet, avec Python 3.12 :

```
python -m unittest discover -v
python bench_q6.py run --timeout 60
python bench_q6.py report
```

La première commande vérifie la correction. La deuxième recrée les instances Meowdoku et le CSV des mesures ; la troisième affiche la synthèse et régénère les tableaux et les données des courbes. Pour répéter uniquement les 50 grilles Meowdoku sans écraser la campagne principale :

```
python bench_q6.py run --suite meowdoku --repeats 3 \
  --csv results/q6_meow_repeats.csv
python bench_q6.py report --csv results/q6_meow_repeats.csv
```

Les répétitions conservent la même graine : elles mesurent la variabilité temporelle, pas celle des choix aléatoires. Les durées et les compteurs d'une seule résolution sont accessibles avec :

```
python sat.py instance.cnf --mode optimized --output-model --stats
```